

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY  
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY



**SOICT**

**Big Data Storage and Processing**

Course: IT4043E

**Capstone Project Report**  
**Weather Monitoring and Forecasting System**

| <i>Group 8</i>      | <i>Student ID</i> |
|---------------------|-------------------|
| Nguyen Tri Thanh    | 20225457          |
| Phan Tran Viet Bach | 20225435          |
| Le Nhat Quang       | 20225522          |
| Nguyen Hoang Son    | 20225525          |
| Nguyen Minh Quan    | 20225520          |

*Instructor*  
Tran Viet Trung, Ph.D.

Hanoi, January 2026

## Table of Contents

|           |                                                                     |           |
|-----------|---------------------------------------------------------------------|-----------|
| <b>1.</b> | <b>Introduction .....</b>                                           | <b>3</b>  |
| 1.1.      | <b>Problem Statement.....</b>                                       | <b>3</b>  |
| 1.2.      | <b>Motivation .....</b>                                             | <b>3</b>  |
| <b>2.</b> | <b>Data Source and Description .....</b>                            | <b>3</b>  |
| 2.1.      | <b>Data Source.....</b>                                             | <b>3</b>  |
| 2.2.      | <b>Data Description .....</b>                                       | <b>4</b>  |
| <b>3.</b> | <b>System Architecture .....</b>                                    | <b>6</b>  |
| <b>4.</b> | <b>Implementation Details .....</b>                                 | <b>7</b>  |
| 4.1.      | <b>Data Ingestion .....</b>                                         | <b>7</b>  |
| 4.2.      | <b>Data Storage.....</b>                                            | <b>10</b> |
| 4.3.      | <b>Spark Processing .....</b>                                       | <b>11</b> |
| 4.3.1     | Batch Layer .....                                                   | 11        |
| 4.3.2     | Speed Layer .....                                                   | 15        |
| 4.4.      | <b>Serving Database .....</b>                                       | <b>16</b> |
| 4.5.      | <b>Visualization Layer .....</b>                                    | <b>17</b> |
| 4.5.1     | Grafana .....                                                       | 17        |
| 4.5.2     | Mapbox App .....                                                    | 19        |
| 4.6.      | <b>Deployment Strategy.....</b>                                     | <b>20</b> |
| <b>5.</b> | <b>Lessons Learned .....</b>                                        | <b>21</b> |
| 5.1.      | <b>Spark Processing Optimization.....</b>                           | <b>21</b> |
| 5.2.      | <b>Version conflict across project by different teammates .....</b> | <b>22</b> |
| 5.3.      | <b>Selecting an Iceberg catalog .....</b>                           | <b>22</b> |
| 5.4.      | <b>Connections to databases.....</b>                                | <b>23</b> |
| 5.5.      | <b>Using existing Helm charts .....</b>                             | <b>23</b> |
| <b>6.</b> | <b>Conclusion .....</b>                                             | <b>24</b> |
|           | <b>Member Contributions.....</b>                                    | <b>24</b> |

# 1. Introduction

## 1.1. Problem Statement

Modern weather monitoring relies heavily on IoT stations that generate vast amounts of meteorological data, but this raw telemetry is often plagued by sensor noise, missing timestamps, and transmission errors. For a Data Science application, this presents a significant challenge: raw environmental data is unsuitable for direct predictive modeling without extensive cleaning and repair. The core problem this project addresses is the difficulty of transforming high-velocity, unstructured sensor readings into a reliable dataset for forecasting. A standard data pipeline often fails to meet the dual requirements of a weather system, which needs to provide both immediate updates for real-time monitoring and rigorously corrected historical data for training accurate machine learning models.

## 1.2. Motivation

The motivation for developing this Weather Monitoring and Forecasting System lies in the growing need for accurate, hyper-local climate insights. While collecting data is easy, bridging the gap between raw ingestion and actionable weather predictions requires a sophisticated engineering approach. We designed this system using the Lambda Architecture to effectively handle these conflicting demands. This architectural choice allows us to decouple our processing logic: a Speed Layer provides near real-time visibility into current conditions, while a parallel Batch Layer utilizes MinIO and Apache Iceberg to build a consistent, high-quality historical record storage system. This setup ensures that we can maintain the data integrity required for downstream forecasting tasks without sacrificing the ability to monitor weather events as they occur.

# 2. Data Source and Description

## 2.1. Data Source

We availed ourselves of the open-source weather API Open-Meteo, which offers free access for non-commercial use with no API key required. More specifically, we utilized the Historical Weather API. This API is based on reanalysis datasets and uses a combination of weather station, aircraft, buoy, radar, and satellite observations to create a comprehensive record of past weather conditions. These datasets are able to fill in gaps by using mathematical models to estimate the values of various weather variables. As a result, reanalysis datasets are able to provide detailed historical weather information for locations that may not have had weather stations nearby, such as rural areas or the open ocean. The table below provides a list of such datasets:

| Dataset                   | Region | Spatial Resolution | Temporal Resolution | Data Availability | Update frequency        |
|---------------------------|--------|--------------------|---------------------|-------------------|-------------------------|
| <a href="#">ECMWF IFS</a> | Global | 9 km               | Hourly              | 2017 – Present    | Daily with 2 days delay |

|                                                    |        |               |          |                |                         |
|----------------------------------------------------|--------|---------------|----------|----------------|-------------------------|
| <a href="#">ERA5</a>                               | Global | 0.25° (~25km) | Hourly   | 1940 – Present | Daily with 5 days delay |
| <a href="#">ERA5-Land</a>                          | Global | 0.1° (~11 km) | Hourly   | 1950 – Present | Daily with 5 days delay |
| <a href="#">ERA5-Ensemble</a>                      | Global | 0.5° (~55 km) | 3-Hourly | 1940 – Present | Daily with 5 days delay |
| <a href="#">CERRA</a>                              | Europe | 5 km          | Hourly   | 1985 – 06/2021 | -                       |
| <a href="#">ECMWF IFS Assimilation Long-Window</a> | Global | 9 km          | 6-Hourly | 2024 – Present | Daily with 2 days delay |

The data dates back to 1940, however, due to the cap of 10,000 daily API calls, we limited ourselves to data from 01/01/2020 to 31/12/2025 across all regions of Vietnam. To balance between the temporal solution and volume, we opted for the hourly data. The details of crawled weather features are presented in the Data Description section.

Conveniently, Open-Meteo offers the sample Python code to setup the API and process the hourly data. Having a list of coordinates, we only modified the code to crawl the data yearly and per location to respect the API. On success, a JSON object will be returned.

## 2.2. Data Description

11 of the most relevant weather variables were selected and described below:

| Variable             | Valid time         | Unit      | Description                                                          |
|----------------------|--------------------|-----------|----------------------------------------------------------------------|
| temperature_2m       | Instant            | °C (°F)   | Air temperature at 2 meters above ground                             |
| relative_humidity_2m | Instant            | %         | Relative humidity at 2 meters above ground                           |
| dew_point_2m         | Instant            | °C (°F)   | Dew point temperature at 2 meters above ground                       |
| precipitation        | Preceding hour sum | mm (inch) | Total precipitation (rain, showers, snow) sum of the preceding hour. |
| cloud_cover          | Instant            | %         | Total cloud cover as an area fraction                                |
| sunshine_duration    | Preceding hour sum | Seconds   | Number of seconds of sunshine of the preceding hour per hour         |
| wind_speed_10m       | Instant            | km/h      | Wind speed at 10 meters above ground                                 |
| wind_direction_10m   | Instant            | °         | Wind direction at 10 meters above ground                             |

|                           |         |           |                                                                       |
|---------------------------|---------|-----------|-----------------------------------------------------------------------|
| weather_code              | Instant | WMO code  | Weather condition as a numeric code                                   |
| soil_temperature_0_to_7cm | Instant | °C (°F)   | Average temperature at the 0-7cm soil level                           |
| soil_moisture_0_to_7cm    | Instant | $m^3/m^3$ | Average soil water content as volumetric mixing ratio at 0-7cm depths |

Here, we feel the need to elaborate on two weather variables: dew point and weather code. First, the weather numeric code follows (World Meteorological Organization) WMO weather interpretation codes. See the table below for more details.

| Code       | Interpretation                                   |
|------------|--------------------------------------------------|
| 0          | Clear sky                                        |
| 1, 2, 3    | Mainly clear, partly cloudy, and overcast        |
| 45, 48     | Fog and depositing rime fog                      |
| 51, 53, 55 | Drizzle: Light, moderate, and dense intensity    |
| 56, 57     | Freezing Drizzle: Light and dense intensity      |
| 61, 63, 65 | Rain: Slight, moderate, and heavy intensity      |
| 66, 67     | Freezing Rain: Light and heavy intensity         |
| 71, 73, 75 | Snow fall: Slight, moderate, and heavy intensity |
| 77         | Snow grains                                      |
| 80, 81, 82 | Rain showers: Slight, moderate, and violent      |
| 85, 86     | Snow showers: Slight and heavy                   |
| 95*        | Thunderstorm: Slight or moderate                 |
| 96, 99*    | Thunderstorm with slight and heavy hail          |

(\*) Thunderstorm forecast with hail is only available in Central Europe

The dew point is the temperature the air needs to be cooled (at constant pressure) to achieve a relative humidity of 100%. At this point, the air can no longer hold more water in the gas form. If it were to be cooled more, water vapor would have to condense into the liquid form, usually as fog or precipitation. The higher the dew point temperature is, the greater amount of moisture is in the air.

In many cases, relative humidity can be misleading because it also measures water vapor, but relative to the air temperature. For example, a temperature of 30°C and a dew point temperature of 30°C will give a relative humidity of 100%, but a temperature of 40°C and a dew point of 30°C produces a relative humidity of 50%. Therefore, the dew point temperature is often chosen over the relative humidity to judge how "dry" or "humid" it feels outside.

Based on these basic weather measures, we can derive other more advanced variables such as vapor pressure, saturation vapor pressure, vapor pressure deficit, and apparent temperature. All of which will be discussed in the Implementation Details section.

### 3. System Architecture

To design a suitable and efficient Big Data system for the Weather Monitoring and Forecasting topic, we adopt the Lambda Architecture, the well-known big data architecture that utilizes both accurate batch processing and real-time stream processing. The following diagram shows the architecture design of our system based on the Lambda Architecture with specific technology choices.

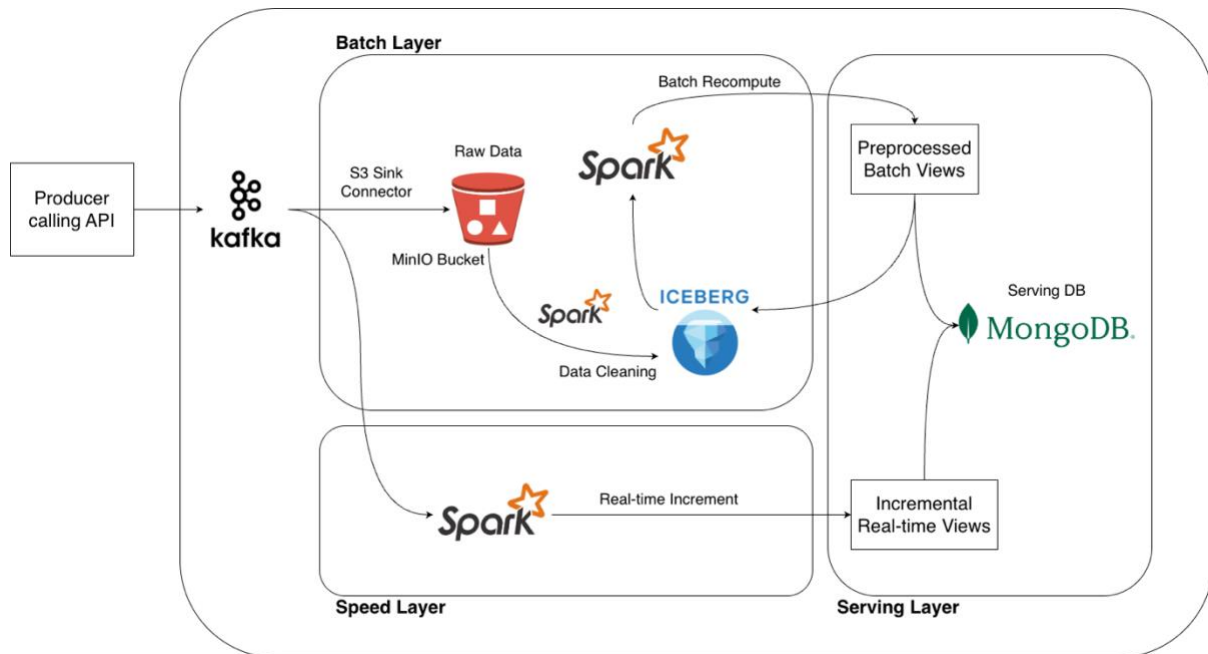


Figure 1: Architecture Overview

We use the following technologies in our system. Along with that, we deploy our system on **Kubernetes**, a container orchestration system widely used to deploy and manage distributed systems, which aligns with our aim to develop a Big Data system capable of scaling up to receive continuous weather data.

- **Apache Kafka**: the message queue component, responsible for receiving data from source and streamlining them to long-term storage and processing
- **MinIO**: S3-compatible object storage system, storing files as objects in buckets
- **Apache Iceberg**: an Open Table Format, allowing to read data files in tabular format along with versioning and schema evolution management through snapshots and manifest lists/files
- **Apache Spark**: data processing engine, capable of both batch and stream processing, and can be deployed as a cluster to make use of computational strengths from multiple machines in a distributed environment
- **MongoDB**: a NoSQL database system, storing records in BSON for efficient reads/writes; in our system, we use MongoDB as the serving database

## 4. Implementation Details

### 4.1. Data Ingestion

The data ingestion stage begins with a **producer** program that performs API calls to the Open-Meteo API to request for data. The HTTP request will take in the location in the form of coordinates (longitude and latitude) along with the date range from which we want to get weather data. Then, we use the Kafka client in Python to produce messages and send to the brokers.

```
def fetch_weather(lat, lon, start_date, end_date):
    url = (
        f"{HISTORICAL_URL}?latitude={lat}&longitude={lon}&start_date={start_date}&end_date={end_date}"
        "&hourly=temperature_2m,relative_humidity_2m,dew_point_2m,precipitation,"
        "cloud_cover,sunshine_duration,wind_speed_10m,wind_direction_10m,weather_code,"
        "soil_temperature_0_to_7cm,soil_moisture_0_to_7cm&timezone=UTC"
    )
    try:
        response = requests.get(url, timeout=60)
        response.raise_for_status()
        return response.json()
    except Exception as e:
        print(f"Failed to fetch data: {e}")
        return None
```

Figure 2: The `fetch_weather` function that makes API calls to Open-Meteo

We deploy the Kafka cluster on Kubernetes using the Strimzi operator. Strimzi allows for seamless Custom Resource Definitions (CRDs) to quickly wind up a Kafka cluster. To fit into our limited resources on a personal device, we simply use 1 Kafka node that acts as both the broker and the cluster controller in KRaft mode. Using KRaft mode removes the need for the traditional Zookeeper dependency. Strimzi also allows specifying roles for groups of nodes, called *node pool*, within a cluster.

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool
# used for specifying pool of Kafka pods with roles
# replicas: num of pods with this config
# roles: broker only, controller only, both for dual-role
# labels: strimzi.io/cluster: <kafka-resource-name>: pods will follow Kafka resource configs under this name
metadata:
  name: dual-role
  namespace: bigdata-pipeline
  labels:
    strimzi.io/cluster: kafka-cluster
spec:
  replicas: 1
  roles:
    - controller
    - broker
  storage:
    type: jbod
    volumes:
      - id: 0
        type: persistent-claim
        size: 1Gi
        deleteClaim: false
        kraftMetadata: shared
```

Figure 3: The `KafkaNodePool` resource

The main topic used for storing messages is named *openmeteo-data*, configured with 3 partitions. Messages will be positioned in partitions based on location names; this allows messages to be separated efficiently since we crawl data at multiple sites. Communications to the cluster are made available through the 3 exposed ports: 9092 for internal Kubernetes cluster access without TLS, 9093 for TLS access, and 9094 for external access from outside Kubernetes, such as from local machine, through the LoadBalancer service.

```
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  # a KafkaNodePool use `name` to get configs for Kafka nodes it create
  # Kafka nodes will be created with this name
  # specify for topics manifests to create such topics in nodes with given name
  name: kafka-cluster
  namespace: bigdata-pipeline
spec:
  kafka:
    version: 4.1.1
    metadataVersion: 4.1-IV1
    listeners:
      - name: plain
        port: 9092
        type: internal
        tls: false
      - name: tls
        port: 9093
        type: internal
        tls: true
      - name: external
        port: 9094
        type: loadbalancer
        tls: false
    config:
      offsets.topic.replication.factor: 1
      transaction.state.log.replication.factor: 1
      transaction.state.log.min.isr: 1
      default.replication.factor: 1
      min.insync.replicas: 1
  entityOperator:
    topicOperator: {}
    userOperator: {}
```

Figure 4: CRD for Kafka cluster with exposed ports for accessing to brokers

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: openmeteo-data
  namespace: bigdata-pipeline
  labels:
    strimzi.io/cluster: kafka-cluster # metadata.name of kafka cluster
spec:
  partitions: 3
  replicas: 1 # adjust based on num. of brokers, for small-scale demo use 1
  config:
    retention.ms: 7200000 # time that events are stored by brokers; after this duration, clear
    segment.bytes: 52428800 # segmentation size of topic data; here, 500Mi
```

Figure 5: CRD for the main topic used, openmeteo-data



The main consumer of messages stored by Kafka brokers will consume messages and store them in the storage backend of our system. Rather than designing a consumer application using the Kafka Python client, we utilize Kafka Connect to streamline messages efficiently. Kafka Connect works by connecting between Kafka and data sources and sinks, with many pre-built *connectors* to and from popular data storages. In our use case, we use the **S3 Sink Connector**, which we will use to *connect* Kafka to our MinIO storage backend. The Kafka Connect cluster is configured to communicate with the Kafka cluster for messages from topics, and the Connector resource specifies the actual connector instance that will create tasks to forward messages from topics to storage sink. Most notably, we use a flush size of 100 and a rotation interval of 5 minutes, meaning the S3 Sink Connector will save messages as files in MinIO every 100 messages consumed or after every 5 minutes. This ensures all messages are consumed at least once, and with offset information which is stored in a Kafka topic, the Kafka Connect cluster ensures that a connector can catch up to the offset position it dropped off in case of failure; together, this enables *exactly-once* consumption of messages through the use of a simple plug-and-play component to Kafka.

```

apiVersion: kafka.strimzi.io/v1
kind: KafkaConnect

metadata:
  name: s3-sink-connect-cluster
  namespace: bigdata-pipeline
  annotations:
    strimzi.io/use-connector-resources: "true"

spec:
  version: 4.1.1
  image: bachthetrollface/kafka-connect-s3:latest # pre-built image with s3 sink connector plugin
  replicas: 1
  bootstrapServers: kafka-cluster-kafka-bootstrap:9093
  groupId: connect-group
  configStorageTopic: connect-configs
  statusStorageTopic: connect-status
  offsetStorageTopic: connect-offsets
  tls:
    trustedCertificates:
      - secretName: kafka-cluster-cluster-ca-cert
        pattern: "*.crt"
  config:
    # -1 means it will use the default replication factor configured in the broker
    config.storage.replication.factor: -1
    offset.storage.replication.factor: -1
    status.storage.replication.factor: -1

    key.converter: org.apache.kafka.connect.storage.StringConverter
    value.converter: org.apache.kafka.connect.json.JsonConverter
    key.converter.schemas.enable: false
    value.converter.schemas.enable: false

```

Figure 6: The Kafka Connect cluster CRD

```

apiVersion: kafka.strimzi.io/v1
kind: KafkaConnector

metadata:
  name: minio-sink-connector
  namespace: bigdata-pipeline
  labels:
    strimzi.io/cluster: s3-sink-connect-cluster

spec:
  class: io.confluent.connect.s3.S3SinkConnector
  tasksMax: 3
  config:
    topics: openmeteo-data

    s3.region: "us-east-1"
    s3.bucket.name: raw-zone
    s3.endpoint: http://minio-internal-service:9000
    s3.path.style.access: true
    s3.force.path.style: true
    store.url: http://minio-internal-service:9000
    path.style.access.enabled: true

    # s3 credentials
    aws.access.key.id: minioadmin
    aws.secret.access.key: minioadmin123

    storage.class: io.confluent.connect.s3.storage.S3Storage
    format.class: io.confluent.connect.s3.format.json.JsonFormat

    key.converter: org.apache.kafka.connect.storage.StringConverter
    key.converter.schemas.enable: false
    value.converter: org.apache.kafka.connect.json.JsonConverter
    value.converter.schemas.enable: false

    flush.size: 100
    rotate.interval.ms: 300000

    schema.compatibility: NONE

```

*Figure 7: The S3 Sink Connector CRD*

## 4.2. Data Storage

We use MinIO as the storage backend for our system. MinIO is an S3-compatible object storage system, popular for its seamless setup and capability to allow a system to upscale its storage to using AWS S3 buckets. Again, to fit into our resources, we use a Persistent Volume Claim (PVC) of 1 GB for MinIO and deploy it with 1 replica. MinIO exposes ports 9000 for API access and 9001 for the web UI console, which users can use to manually view and check stored files. To further make use of the console, we expose it using a LoadBalancer service and have our messages saved in JSON format, allowing us to manually check for data validity when saved using Kafka Connect.

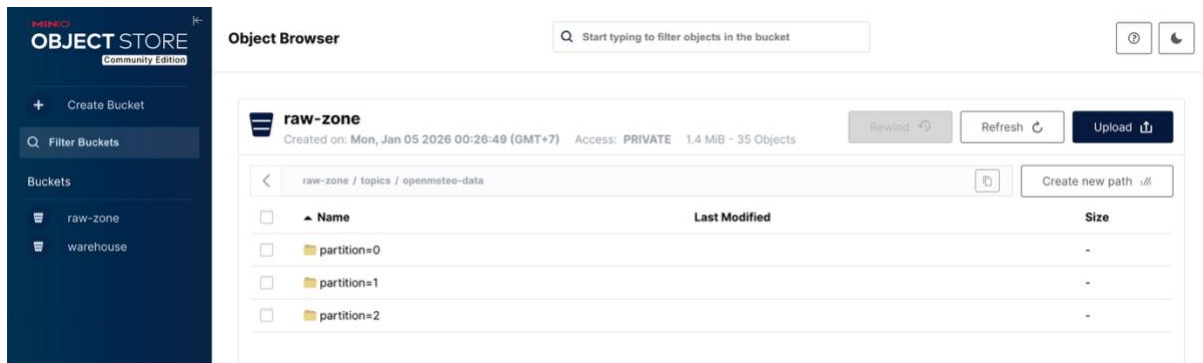


Figure 8: The MinIO web console. The raw-zone bucket stores raw data saved using S3 Sink Connector, partitioned into directories based on partitions in Kafka topic

We also use Apache Iceberg as the table format for efficient data storage in the form of tables, as well as its ability to handle schema evolution and time travel queries. Specifically, Iceberg requires the use of a catalog to keep track of tables’ metadata. In our system, we use the Nessie catalog, a catalog with Git-like features such as branches for version control. However, we chose Nessie with the main reason that it acts as a centralized service for accessing tables, ensuring secure read-write operations. Nessie uses a database for storage of “commits” to store tables’ metadata in the long term; it supports a wide range of databases for this version-store purpose, and we selected MongoDB as we will also be using this database in our system.



Figure 9: The Nessie web UI for viewing tables, along with the option to view “commit history” made to tables and to checkout to different branches

### 4.3. Spark Processing

In this section, we will mention more details on how we perform transformations on our selected data source. Implementation for these transformations using PySpark can be found in our source code under the directory *src/spark*.

#### 4.3.1 Batch Layer

With the goal of incrementally and progressively improving the structure and quality of data, our batch layer is designed to follow the medallion architecture. The Bronze layer is where we land all the raw data from external sources. As previously mentioned, Kafka will read data records directly from the Open-Meteo API and write to MinIO. In the Silver layer, data from the Bronze layer is correctly converted, transformed, and cleansed to a minimal degree to prioritize speed and agility. Finally, the already cleaned data flows to the Gold layer, where advanced feature engineering is carried out to extract “model-consumption-ready” data features.

#### 4.3.1.1 Bronze-to-Silver Transformation

The data transition from the Bronze layer (raw ingestion) to the Silver layer (refined data) is the core data cleaning and quality assurance step in our pipeline. This process ensures that any downstream usages receive consistent, physically valid, and continuous time-series data. The transformation could be divided into four stages.

##### Stage 1: Deduplication

To ensure strict time-series integrity of one observation per hour per city, all duplicated records based on the composite keys of [timestamp, city] are dropped.

##### Stage 2: Type Conversion, String Standardization, and Outlier Nullification

**Type conversion:** To avoid any caveats in schema imposing, timestamp columns such as timestamp and ingested\_at were first declared to be of StringType(). It is incumbent that they are converted to TimestampType() as they will be used in several complexed data transactions down the line.

**String standardization:** String columns underwent the following normalizations to remove formatting errors and code readability.

- **Lowercasing:** Hà Nam-Nam Định (tây) → hà nam-nam định (tây)
- **Accent removal:** hà nam-nam định (tây) → ha nam-nam dinh (tay)
- **Parentheses removal:** ha nam-nam dinh (tay) → ha nam-nam dinh tay
- **Slugification:** ha nam-nam dinh tay → ha\_nam\_nam\_dinh\_tay

##### Outlier nullification:

Numerical weather data will then be validated against know scientific bounds. Values outside of these ranges are converted to NULL rather than being capped to prevent artificial bias.

| Variable             | Valid Range | Unit      | Scientific basis                                                                                                                                      |
|----------------------|-------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| temperature_2m       | -95 to 65   | °C (°F)   | Covers Earth's historical extremes: Lowest (-89.2°C, Vostok) and Highest (56.7°C, Death Valley)                                                       |
| relative_humidity_2m | 0 to 105    | %         | 5% margin for sensor calibration error                                                                                                                |
| dew_point_2m         | -95 to 50   | °C (°F)   | Physically, dew point can't exceed air temperature. The upper bound (50°C) is set near the theoretical maximum limit of atmospheric moisture capacity |
| precipitation        | 0 to 400    | mm (inch) | Safely covers the world record for 1-hour rainfall (~305mm in Holt, MO, 1947)                                                                         |
| cloud_cover          | 0 to 100    | %         | Percentage                                                                                                                                            |
| sunshine_duration    | 0 to 3600   | Seconds   | Maximum seconds in one hour is 3600                                                                                                                   |
| wind_speed_10m       | 0 to 410    | km/h      | Upper bound covers the highest non-tornadic wind gust ever recorded                                                                                   |

|                           |               |                                |                                                                                                                                  |
|---------------------------|---------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
|                           |               |                                | (408 km/h during Cyclone Olivia)                                                                                                 |
| wind_direction_10m        | 0 to 360      | °                              | Standard meteorological compass degrees (0° = North, 90° = East).                                                                |
| soil_temperature_0_to_7cm | -50 to 75     | °C (°F)                        | Soil has high thermal inertia and rarely fluctuates as wildly as air temperature. However, desert surface soils can exceed 60°C. |
| soil_moisture_0_to_7cm    | -0.05 to 1.05 | m <sup>3</sup> /m <sup>3</sup> | Volumetric water content is ratio-based (0-1). We allow slight negatives (-0.05) to account for sensor noise in dry soil.        |

### Vector Decomposition: Wind direction

Wind direction is circular, where 359° is physically far from 1°, but physically adjacent. Without this special handling, any aggregations will be mathematically wrong whenever the wind blows from the North. For example, average wind direction between 350° (North – North - West) and 10° (North – North - East) will result in 180 (South).

$$\theta_{rad} = \theta_{deg} \times \frac{\pi}{180} \text{ where } \text{sine\_component} = \sin(\theta_{rad}) \text{ and } \text{cosine\_component} = \cos(\theta_{rad})$$

By breaking the direction into its Sine and Cosine components, we avoid aggregating angles and instead aggregating on vectors. These vectors can later on be recomposed back to angles.

### Stage 3: Imputation

We employed a hybrid two-stage imputation strategy to handle missing values (NULLs) caused by data inconsistency or outlier removal.

#### Weighted Moving Average (WMA)

This is the primary imputation method with the aim to preserve local trends and smoothness. It looks at the 3-hour data records surrounding the missing value where closer neighbors are assigned higher weights (higher influence) where radius 1: weight 3, radius 2: weight 2, and radius 3: weight 1

$$\text{Value} = \frac{\sum(\text{Neighbor Value} \times \text{Weight})}{\sum \text{Existing Weights}}$$

#### Global Fill

If the WMA fails (a NULL data gap > 7 hours), we use a fallback mechanism.

- **Forward Fill** (Last Observation Carried Forward): Assumes weather persists over time
- **Backward Fill**: Used only for gaps at the very start of the dataset
- **Default** (-100): The absolute last resort

The window frames used for Forward/Backward Fill are strictly limited to 24 hours to limit the search space and thus reduce the memory overhead on Spark

Executors.

#### Stage 4: Finalization

**Vector Recomposition:** The imputed Sine and Cosine components are converted back into degrees and normalized to the  $[0, 360]$  range

$$\theta_{\text{final\_rad}} = \text{atan2}(\text{cosine\_component}, \text{sine\_component})$$

$$\theta_{\text{final\_deg}} = \theta_{\text{final\_rad}} \times \frac{180}{\pi}$$

$$\theta_{\text{norm}} = (\theta_{\text{final\_deg}} + 360) \bmod(360)$$

**Cleanup:** Temporary vector columns are dropped

With this transformation pipeline, we have a clean, continuous, and scientifically valid datasets ready for the Apache Iceberg storage layer and further complexed transformations.

##### 4.3.1.2 Silver-to-Gold Transformation

The Gold layer transformation takes the cleaned, continuous time-series data from the Silver layer and enriches it with advanced weather features designed specifically to forecast **temperature\_2m**. The transformation process involves four key steps.

#### Step 1: Cyclical Time Encoding

Similar to `wind_direction_10m`, timestamp columns are also circular. Machine Learning models often struggle with such raw cyclical features because they interpret them linearly. With the same inspiration, we will project time features onto a unit circle using sine and cosine transformations.

$$x_{\sin} = \sin\left(2\pi \cdot \frac{t}{\text{period}}\right) \quad x_{\cos} = \cos\left(2\pi \cdot \frac{t}{\text{period}}\right)$$

With these, we have

- **Hour of Day:** Encoded as **hour\_sin** and **hour\_cos**
- **Day of Year:** Encoded as **day\_of\_year\_sin** and **day\_of\_year\_cos**

#### Step 2: Autoregressive (Lag) Features

Weather forecasting relies heavily on the **Persistent Model** – the assumption that the future state of the atmosphere is strongly correlated with its recent past. We bring this historical context to our Gold layer data by generating lag features for **temperature\_2m** and **dew\_point\_2m** at the following intervals:

- **1 hour, 2 hours, 3 hours:** To capture immediate short-term patterns
- **24 hours:** To capture the daily diurnal cycle

#### Step 3: Rolling statistics (Trend Discovery)

To better understand the slowly changing climate state, we calculate the rolling

**Mean and Standard Deviation** for key variables such as **temperature\_2m**, **relative\_humidity\_2m**, **dew\_point\_2m**, **wind\_speed\_10m**, and **precipitation**. The following time windows were selected:

- **12 hours:** Half-day trends (day/night transition)
- **24 hours:** Daily trends
- **3 days:** Multi-day weather movement
- **1 week:** Weekly seasonality and longer-term shifts

#### Step 4: Advanced Meteorological Features

Since weather dynamics are influenced by the combined effects of several factors, we conducted a small literature review and subsequently engineered additional features to capture interactions among the original weather variables.

- **Vapor Pressure:** Represents the partial pressure exerted by water vapor in the air. Unlike Relative Humidity, this is an absolute measure of moisture independent of temperature changes.

$$e_a = 6.112 \times \exp\left(\frac{17.67 \times T_{dew}}{T_{dew} + 243.5}\right) (hPa)$$

- **Saturation Vapor Pressure:** The maximum pressure of water vapor the air can hold at the current temperature before saturation occurs

$$e_s = 6.112 \times \exp\left(\frac{17.67 \times T_{air}}{T_{air} + 243.5}\right) (hPa)$$

- **Vapor Pressure Deficit (VPD):** The difference between how much moisture the air can hold and how much it actually holds. Basically, it measures the "drying power" of the air. The higher the VPD, the more "thirsty" the air becomes, the more moisture it pulls from soil and plant.

$$e_s - e_a$$

- **Apparent Temperature (AT):** A combined index to estimate how the temperature feels to a human. It accounts for both the muggy effect of humidity and the cooling effect of wind.

$$AT = T_{air} + 0.33 \times e_a - 0.70 \times V_{wind} - 4$$

The generation of lagging and rolling features creates NULL values at the beginning of the dataset. To ensure data quality, we drop the first week of records where these features are undefined. This enrichment provides a more rigorous physical representation of atmospheric dynamics—specifically capturing moisture capacity and thermal stress—thereby reducing the need to induce these complex non-linear relationships from raw telemetry.

### 4.3.2 Speed Layer

As depicted in our architecture model, the Speed Layer operates in parallel to the Batch Layer, designed to ingest and process real-time data streams with minimal latency. While the Batch Layer focuses on comprehensive historical accuracy, the

Speed Layer prioritizes immediacy, providing a "current raw truth" to the serving layer.

Ideally, the transformation logic in the Speed Layer should mirror the Bronze-to-Silver transformation to ensure consistency across views. However, given the constraints of Structured Streaming and the requirement for low latency, we employed a streamlined version of that pipeline. Specifically, we omitted computationally expensive stateful operations such as **Wind Vector Decomposition** and **WMA Imputation**, as the latter relies on future data points (Lead functions) which are unavailable in a real-time stream.

The transformation process is divided into two primary stages:

### **Stage 1: Casting, Standardization, and Nullification (Stateless Transformation)**

To maintain data hygiene, we replicated the stateless cleaning steps from the Bronze-to-Silver transformation.

- **Type conversion:** To ensure schema validity for downstream consumers, it is incumbent that timestamp columns **timestamp** and **ingested\_at** are cast from **StringType()** to **TimestampType()** immediately upon ingestion.
- **String standardization:** We applied the same normalization rules—lowercasing, Vietnamese accent removal, and slugification—to ensure text consistency.
- **Outlier Nullification:** Real-time values are validated against the same scientific bounds used in the batch layer. Values falling outside these physically valid ranges are converted to NULL to prevent skewed analytics in the real-time views.

### **Stage 2: Streaming Deduplication with Watermark**

In the Batch Layer, deduplication is straightforward as we have access to the entire finite dataset. However, in a streaming context where data is theoretically infinite, maintaining the state of every unique key forever would eventually crash the system memory.

To ensure strict integrity while managing resource constraints, we implement **Watermarking** alongside our deduplication logic. We define a watermark of **5 minutes** on the **ingested\_at** column. This mechanism instructs the Spark engine to maintain deduplication state only for data arriving within this time window, allowing it to safely discard state for older records and preventing memory saturation. Records arriving after this watermark threshold are ignored, ensuring the system remains performant.

## **4.4. Serving Database**

MongoDB is selected for the serving layer due to its apparent convenience in storing unstructured data and efficient read/write operations. We provide a 1GB PVC for Mongo to ensure sufficient storage in development conditions and expose the port 27017 using a NodePort service for other components to access. Additionally, we include the Mongo Express application, the web UI for MongoDB. We make use of



Mongo Express to manually check for status of data. Also, as mentioned above, we also make use of MongoDB as the version store for Nessie catalog by using a different database named *nessie*, separating the metadata storage from the actual processed data.

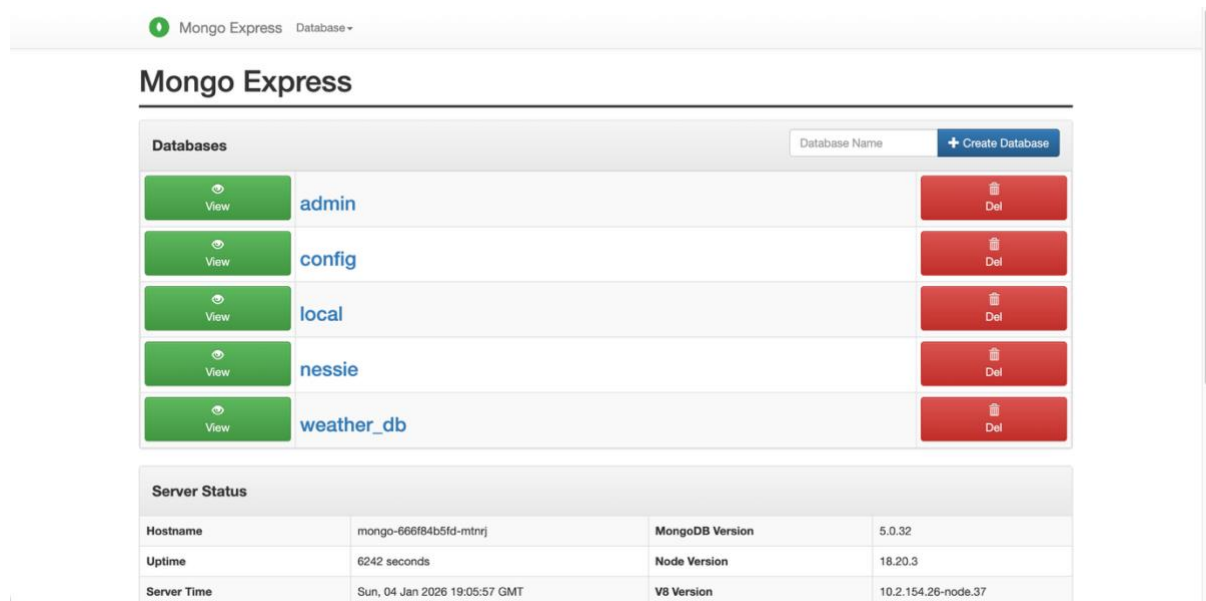


Figure 10: The Mongo Express UI, featuring the *weather\_db* database that stores our processed data and the *nessie* database that stores version metadata for Nessie

## 4.5. Visualization Layer

### 4.5.1 Grafana

The visualization layer is engineered to provide comprehensive observability into both the meteorological data pipeline and the underlying system infrastructure. This subsystem employs a cloud-native observability stack centered around **Prometheus** for metric ingestion, and **Grafana** for multi-dimensional visualization. Data is exposed to the monitoring system via two distinct exporters, targeting separate observational domains:

**Weather Exporter.** A custom Python-based exporter was developed to bridge the gap between the persistence layer (MongoDB) and the time-series database. This component specifically targets the Gold Layer of the data pipeline, which contains highly processed, scientifically enriched meteorological features. Validated features from the Gold schema are mapped to Prometheus Gauge metrics, including:

- **Physical Indicators:** *temperature\_2m*, *humidity*, *precipitation*.
- **Thermodynamic Indices:** *apparent\_temperature* (feels-like), *vapor\_pressure\_deficit* (dryness risk).
- **Statistical Trends:** Rolling means and standard deviations calculated during the Silver-to-Gold transformation.

**Node Exporter.** To monitor the health of the host environment, the standard Node Exporter is deployed. This component collects hardware and OS-level metrics, including CPU utilization, memory pressure, disk I/O throughput, and network

traffic. This ensures that the computational resources running the ETL pipelines are operating within optimal parameters.

**Metric Ingestion and Historical Management.** The ingestion layer is configured to ensure high availability and data longevity. Prometheus is configured via Kubernetes ConfigMap, polls both the *Weather Exporter* and *Node Exporter* at a defined 15-second interval.

**Visualization Layer (Grafana).** Grafana is provisioned as the presentation layer, connecting to the Prometheus as its primary Data Source. This configuration provides a unified query interface for both real-time and historical data. Visualization is organized into two dedicated dashboards: Meteorological Dashboard and System Health Dashboard

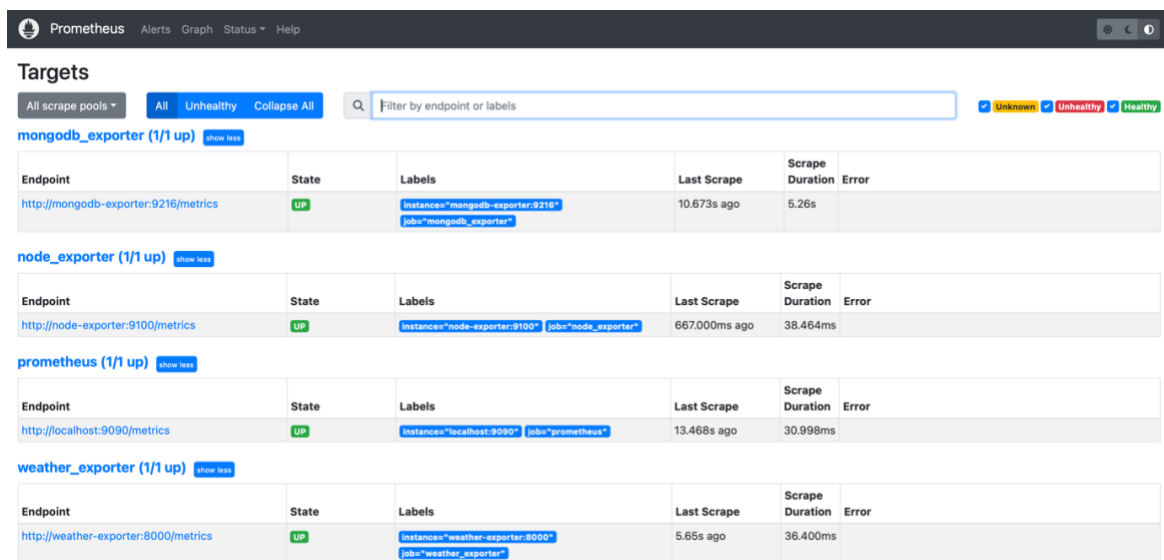


Figure 11: The Prometheus Targets Dashboard, showing healthy connections to data sources

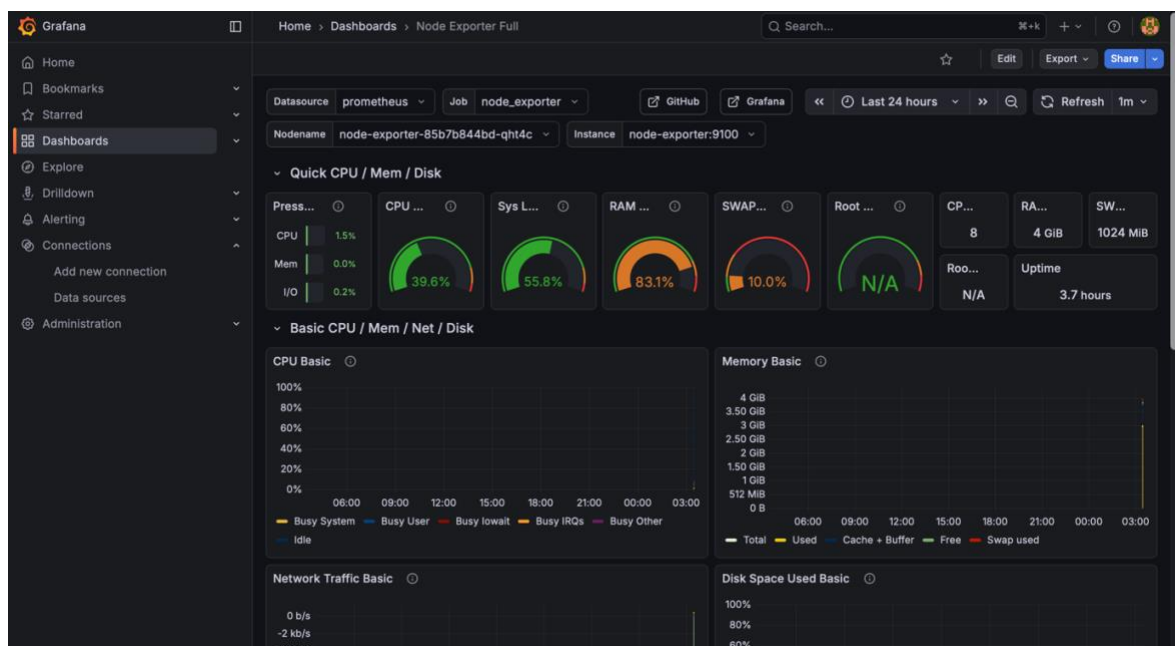


Figure 12: The Grafana Dashboard showing system metrics (CPU usage, Memory usage, etc.)

### 4.5.2 Mapbox App

The real-time weather visualization component of the system is built upon **Mapbox GL JS**, a JavaScript library that uses WebGL to render interactive maps from vector tiles and Mapbox styles. This engine was selected for its high-performance rendering capabilities, particularly when handling dynamic geospatial data such as wind vectors and meteorological heatmaps.

Unlike traditional raster-based mapping libraries, Mapbox GL JS allows for client-side rendering, enabling smooth transitions, rotation, and dynamic styling of map elements without requiring server-side image generation. The system utilizes a custom style layer to overlay meteorological data (temperature, rainfall, wind speed) onto a geographical base map.

The application follows a containerized Microservices architecture managed via Docker. The codebase is organized into a clear separation of concerns between the Backend (Data Processing/Serving) and the Frontend (Visualization).

#### Directory Structure:

```
mapbox/
├── Dockerfile           # Node.js environment definition
├── backend/             # Server-side Logic
│   ├── server.js        # Entry point, API routing, and dynamic env injection
│   ├── dataLoader.js    # Data retrieval, transformation, and database interaction
│   ├── models.js        # MongoDB Schema definitions and Object Mapping
│   └── utils.js         # Helper functions (date processing, logic utilities)
├── frontend/           # Client-side Visualization
│   ├── index.html       # Main entry for the web interface
│   ├── css/             # Stylesheets for UI components
│   └── js/
│       ├── main.js      # Mapbox initialization and event listeners
│       ├── api.js       # Asynchronous data fetching from Backend
│       └── config.js    # Visualization parameters and color scales
```

#### Backend Implementation

The backend serves as the bridge between the persistent storage (MongoDB) and the visualization layer.

- **Backend entrypoint (server.js):** The module functions as the central entry point for the application backend, implemented using the **Express.js** framework. Its primary responsibilities include initializing the database connection, defining API endpoints, and serving static frontend assets.
- **Data Transformation (dataLoader.js):** This module implements an Adapter pattern. It retrieves raw meteorological data from MongoDB and transforms the field names to match the frontend's expected schema. This ensures the frontend remains decoupled from changes in the database schema.
- **Object Modeling (models.js):** utilized Mongoose to define strict schemas for data validation. The model is explicitly mapped to the specific MongoDB

collection defined via the WEATHER\_COLLECTION\_NAME environment variable.

## Frontend Implementation

The frontend is responsible for rendering the visual representation of the data.

- **Map Initialization:** The map is instantiated with specific constraints, including bounds to limit the user's view to the relevant geographical area and maxZoom/minZoom to control the level of detail.
- **Multiple View:** The system supports 4 views, including windy map, temperature map, precipitation map, and cloud coverage map.
- **Data Interpolation:** The system uses Mapbox's expression language (e.g., `['interpolate', ['linear'], ['get', 'value'], ...]`) to create smooth visual gradients for temperature and rainfall, mapping numerical values to continuous color spectra.

The visualization behavior is governed by a centralized configuration module (config.js). These parameters allow for fine-tuning the visual output without altering the core rendering logic.

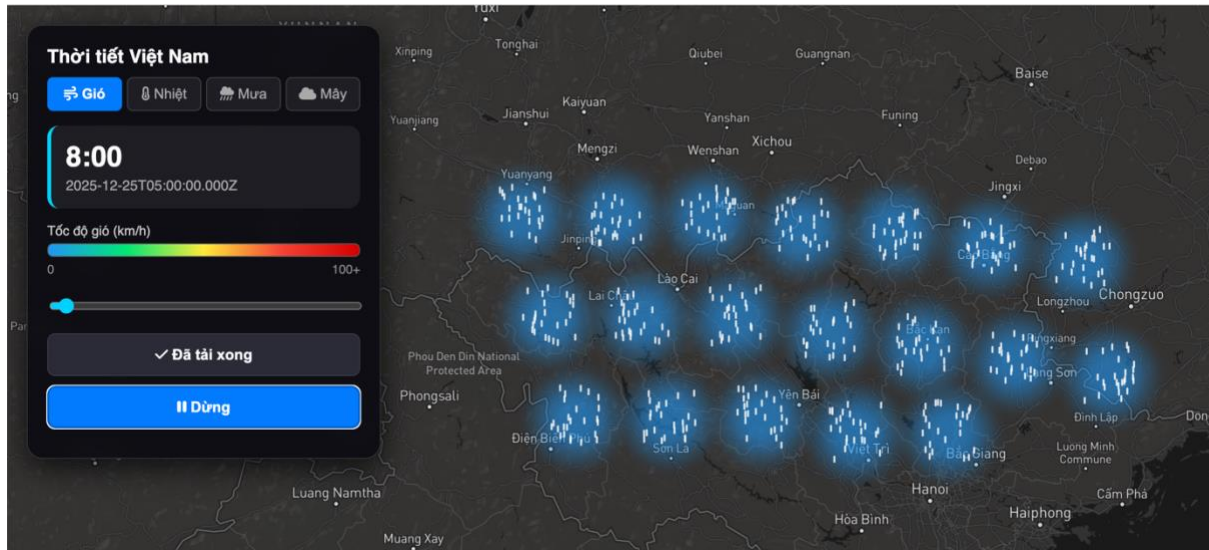


Figure 13: The Mapbox visualization application, visualizing wind directions and magnitudes

## 4.6. Deployment Strategy

We carry out development and deployment of the system in 3 stages:

- **Development and Testing** of individual components. We individually experiment with components using Docker.
- **Integration of Components.** We integrate groups of components (e.g. data ingestion, storage, etc.) using Docker Compose to verify that they work well together.
- **Kubernetes Migration.** In the final stage, we move components part by part into Kubernetes, using Deployments for stateless components, StatefulSets for

storage components, Services to enable connectivity, and Helm charts for pre-built components for seamless deployment (here, the Strimzi operator).

As we develop and deploy components, we come together and keep track of versions of images and packages used in the system. This ensures we do not encounter version mismatch and potentially failure during deployment. Additionally, we use **Minikube** to run Kubernetes locally, allowing seamless setup of a simple Kubernetes cluster with one single node.

## 5. Lessons Learned

### 5.1. Spark Processing Optimization

**Problem:** During the initial development of the batch processing pipeline, we encountered several performance bottlenecks as the complexity of our transformations increased. Most severely, when calculating the "Gold" layer features—which involved extensive window functions for rolling averages and lag features—our Spark jobs frequently suffered from slow execution times and occasional Out-Of-Memory (OOM) errors.

**Solution:** We observed that the driver was struggling to manage the lineage of multiple intermediate DataFrames, and the re-computation of the entire pipeline whenever we triggered a count or save action caused redundant processing overhead. To address this, we did the following improvements:

- **Eliminated the usage of Python loops and slow User-Defined Functions (UDFs)** for string standardization: Replacing them with Spark's built-in SQL functions (**regexp\_replace**, **translate**, etc.). This kept the data processing strictly within the JVM, avoiding the expensive serialization cost of moving data between Java and Python.
- **Adopted a "one-pass" coding style:** By chaining transformations into single select expressions, the creation of intermediate variables were reduced and the Catalyst optimizer could fuse operations more effectively.
- **Implemented strategic caching:** For any processed data frames with further usage
- **Split Spark Medallion Jobs:** Instead of using the cached Silver data immediately as input to the Silver-to-Gold transformation, we wrote the Silver data to Apache Iceberg tables which will be read for the final transformation stage in our batch layer.

**Key takeaway:** We learned that Spark's "lazy evaluation" model requires careful planning; without explicit caching, Spark may blindly re-compute the entire dependency graph when every action is invoked, leading to massive inefficiencies. Additionally, we gained a deeper appreciation for the cost of serialization in PySpark, realizing that native column-based operations are vastly superior to row-by-row Python logic. Optimization is not an afterthought but a critical design phase, where

understanding the underlying physical execution plan is as important as writing the transformation code itself.

## 5.2. Version conflict across project by different teammates

**Problem:** While working in a team, we encountered version incompatibility between Scala, Apache Spark and MongoDB. Although the application executed without errors, no documents were written to the checkpoint, leading to silent data loss.

**Solution:** We investigated the issue and identified the correct compatible versions of Spark and MongoDB by consulting official documentation and community resources, then aligned all team environments accordingly.

**Key takeaway:** Version compatibility is a critical consideration during the planning and pipeline design phase. Aligning dependency versions early can prevent hidden runtime issues and ensure consistent behavior across team members' environments.

## 5.3. Selecting an Iceberg catalog

**Problem:** During deployment of Iceberg, we have to make the decision on which type of catalog to use.

**Solution:** We found 3 catalog types for selection

- **Hadoop catalog.** This is the simplest type of catalog, using directly the file system for storing tables' metadata, meaning that the catalog will trace through directory names to look for metadata of a table. However, we noticed that using Hadoop catalog is insecure as changing a folder's name could make the catalog lose track of the corresponding table.
- **REST catalog.** It provides a REST API for applications to access and read tables, allowing centralized management of tables. Additionally, using such a centralized service for accessing data storage means safer concurrent reads and writes, which is expected to be common for e.g. multiple Spark workers writing to a table. This was our initial selection for the Iceberg catalog, but we failed to deploy it properly and could not make it connect to the MinIO storage backend.
- **Nessie catalog.** As we are considering between attempting to fix the REST catalog and falling back to using Hadoop catalog, we found Nessie as a similar type of catalog to REST. It also exposes a REST API, but processing engines like Spark provides plugin packages to directly communicate to Nessie, meaning it is more convenient compared to REST catalog. Additionally, Nessie provides a Git-like experience to manage tables that can be useful to separate between development and deployment environments, although we did not make use much of this feature.

**Key takeaway:** There can be multiple options for a specific component with tradeoffs between e.g. simplicity in deployment and suitability for production.

## 5.4. Connections to databases

**Problem:** Occasionally we would face issues connecting to MongoDB, from both Nessie and Spark. The issue is always the same: failed authentication

**Solution:** We tried configuring the database credentials at different places:

- For Nessie's Helm chart, we provided credentials to fields in the configuration file during the chart installation. We tried both using Kubernetes Secrets and providing the credentials directly but still failed.
- For Spark, we tried providing the credentials directly in the database URI but output logs still show failed authentication error.

Eventually, we did some research and found that the issue is due to how MongoDB looks for a user's credentials. Mongo would look for the user in the database specified for accessing, e.g. *nessie*, instead of the default *admin* used for authentication. We finally resolved the problem by specifying a parameter in the URI: *?authSource=admin*. This ensures Mongo will use the default *admin* database to check credentials.

**Key takeaway:** Different database systems handle URIs and authentication differently. One should check out carefully official documentations to use URIs and credentials the right way to avoid authentication issues.

## 5.5. Using existing Helm charts

**Problem:** During deployment of Nessie catalog on Kubernetes, we first used its existing Helm chart, as suggested by Nessie's official documentations. However, we failed to connect it to the MongoDB version-storing database. Through viewing logs, we observed that the problem is due to failed authentication.

**Solution:** We tried 3 solutions, out of which only the third one worked:

- We first attempted to hard-code the MongoDB access credentials instead of providing them through Kubernetes Secrets
- We then modify the database URI to include both the credentials and a parameter *authSource=admin*, as suggested by many encountering similar authentication issues. The parameter ensures Mongo will authenticate using credentials stored in the built-in *admin* database instead of in the target *nessie* one.
- When above methods failed, we resort to using a Deployment to set up Nessie with database URI and credentials provided through environment variables of the container, and this resulted in Nessie successfully connecting to MongoDB

**Key takeaway:** There exist many Helm charts that can be used to deploy widely-used applications on Kubernetes but using them requires advanced knowledge on how to configure the components. For simple deployments, and if using Helm charts fail, one can always return to the more traditional way of using Deployments and Services. This approach is also closer to how you need to configure Docker containers, allowing new Kubernetes users to easily adapt to using Kubernetes.

## 6. Conclusion

In this project, we have been able to implement a Lambda Architecture big data processing pipeline deployed on a Kubernetes cluster. We built the pipeline upon a comprehensive technology stack, utilizing Kafka for high-throughput ingestion, MinIO for object storage, and Spark for distributed processing. Furthermore, by integrating Apache Iceberg for efficient table management and Nessie as the centralized catalog, this infrastructure provided a practical environment to understand how these distributed systems interact to bridge the gap between real-time streams and reliable batch analytics.

We applied the Medallion architecture to structure our data transformations, systematically progressing from raw telemetry to a "Gold" state suitable for analysis. This process involved tackling challenges such as schema enforcement, outlier nullification, and feature engineering for machine learning. Working through these stages reinforced our technical understanding of distributed computing and demonstrated the critical importance of data quality when preparing datasets for forecasting models.

While the current system functions as a working prototype, we recognize there is still room for optimization, scalability, and improvement. Future work will include building a Machine Learning model to better evaluate our processed data quality and improving the efficiency of our streaming pipeline. Ultimately, this project served as a comprehensive exercise in connecting big data tools into a cohesive workflow.

### Member Contributions

| Member Name         | Contributions                                                                                                                                                                                                                                                      |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nguyễn Trí Thanh    | <ul style="list-style-type: none"><li>- Researched on the topic for insights on data format and semantics</li><li>- Designed the Mapbox app and Grafana to visualize weather history over time</li></ul>                                                           |
| Phan Trần Việt Bách | <ul style="list-style-type: none"><li>- Researched on the technologies used for the system</li><li>- Designed and implemented the data ingestion stage including Kafka and Kafka Connect, the Nessie catalog, and worked on deploying them on Kubernetes</li></ul> |
| Nguyễn Minh Quân    | <ul style="list-style-type: none"><li>- Designed, implemented and tested the Spark cluster used for data processing</li><li>- Worked on deploying and integrating the system on Kubernetes</li></ul>                                                               |
| Lê Nhật Quang       | <ul style="list-style-type: none"><li>- Researched on and implemented the transformation operations for the weather data using Spark</li></ul>                                                                                                                     |



|                  |                                                                                                                                                                                                     |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | <ul style="list-style-type: none"> <li>- Contributed to implementing and testing the Kafka cluster</li> </ul>                                                                                       |
| Nguyễn Hoàng Sơn | <ul style="list-style-type: none"> <li>- Implemented and tested storage components, including MinIO and MongoDB</li> <li>- Implemented and tested connections from Spark to data storage</li> </ul> |